

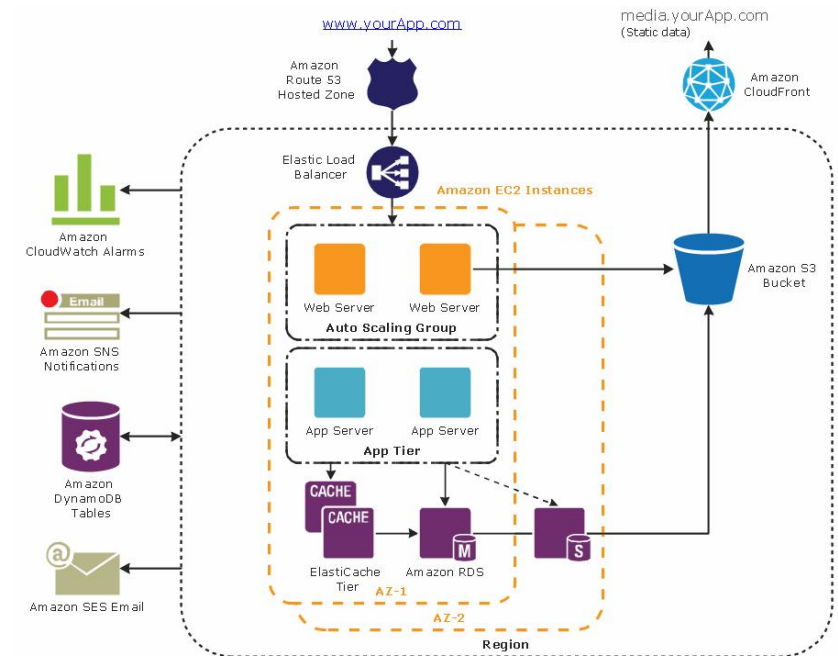
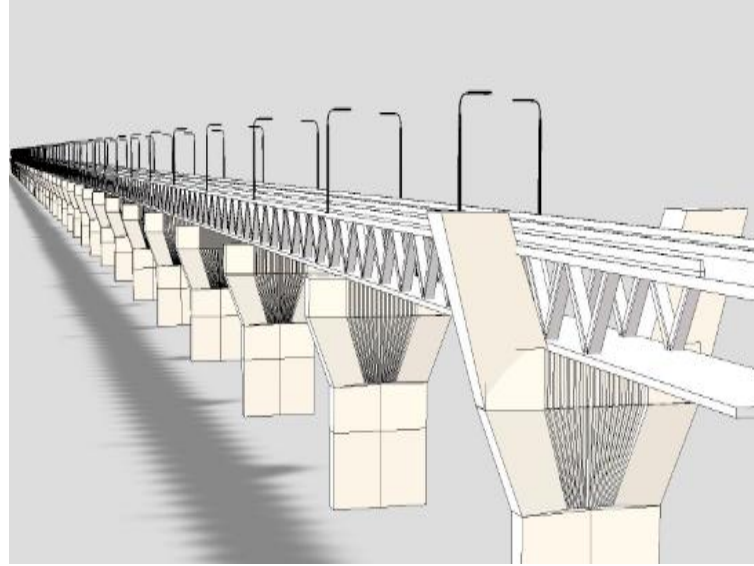


CSE 4513

Lec – 7

Modeling with UML

MODEL



- ✓ A model is a **simplification of reality**.
- ✓ Helps to better understand the system we are developing.
- ✓ Through modeling, we achieve four aims.
 1. to **visualize a system** as it is or as we want it to be.
 2. to **specify the structure or behavior** of a system.
 3. to **provide a template** that **guides us in constructing** a system.
 4. to **document the decisions** we have made.

Creating models before software development is not just a luxury—it's a necessity.

UML BASICS

MODELING PRINCIPLES

WHAT IS MODELING



- ✓ Modeling consists of building an abstraction of reality.
- ✓ Abstractions are simplifications because:
 - They ignore irrelevant details and
 - They only represent the relevant details.
- ✓ What is **relevant** or **irrelevant** depends on the purpose of the model.

WHY MODELING SW



- ✓ Software is getting increasingly more complex
 - Windows XP > 40 million lines of code
 - A single programmer may not manage this amount of code in his entire coding life.
- ✓ Code is not easily understandable by developers who did not write it
- ✓ We need simpler representations for complex systems
 - Modeling is a means for dealing with complexity

WHAT IS UML?



Unified Modeling Language

OMG Standard **Object Management Group**

UML is a modeling language to express and design documents, software

Particularly useful for OO design

Independent of implementation language



WHY USE UML



- ✓ **Visualization:** Makes complex systems easier to understand with graphical representations.
- ✓ **Standardization:** Ensures consistency and provides a common language for developers and stakeholders.
- ✓ **Documentation:** Serves as detailed documentation for future reference and maintenance.
- ✓ **Design and Analysis:** Helps in system design before coding, avoiding issues early.
- ✓ **Better Communication:** Facilitates communication between technical and non-technical stakeholders.
- ✓ **Supports Development Processes:** Fits with different methodologies, including Agile and Waterfall.

UML DIAGRAMS

UML DIAGRAM TYPES



There are two main categories: **structure diagrams** and **behavioral diagrams**.

components

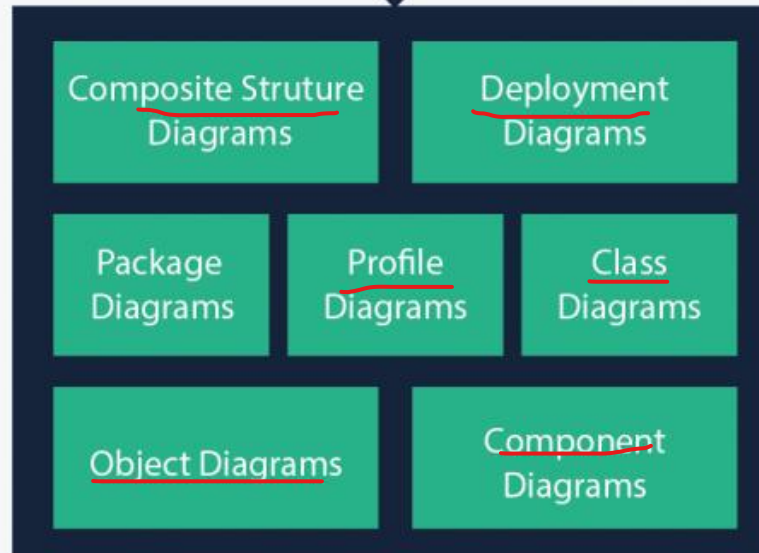
decisions

show the things in the modeled system

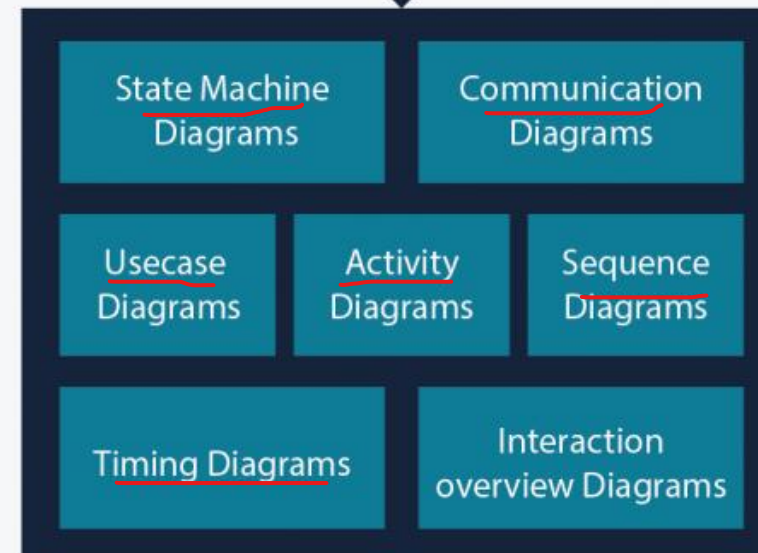
show what should happen in a system

- ✓ are static in nature
- ✓ represents the system's components, their organization, and their relationships.

Structural Diagrams



Behavioral Diagrams



- ✓ are dynamic and
- ✓ emphasize the interactions, state changes, and behavior of the system's components over time.

UML DIAGRAMS



- ✓ Use Case Diagrams
- ✓ Class Diagrams
- ✓ Object Diagrams
- ✓ Package Diagrams
- ✓ Sequence Diagrams
- ✓ Activity Diagrams
- ✓ State Transition Diagrams
- ✓ Deployment Diagrams

USE CASE



- ✓ Used during requirements elicitation to ~~represent external behavior~~
- ✓ The purpose of Use Case diagram
 - To identify functions and how roles interact with them.
 - For a high-level view of the system
 - To identify internal and external factors

USE CASE (CONT'D)

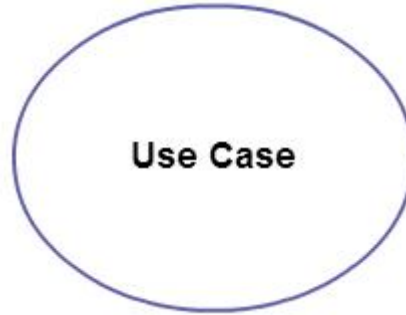


Use case diagrams consist of 4 objects.

1. Actor
2. Use case
3. System
4. Package

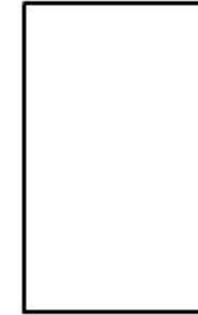


any entity that performs a role in one given system. This could be a person, organization or an external system and usually drawn like skeleton.



represents a function or an action within the system. It's drawn as an oval and named with the function.

System



define the scope of the use case and drawn as a rectangle. This an optional element but useful when you're visualizing large systems.

Package Name



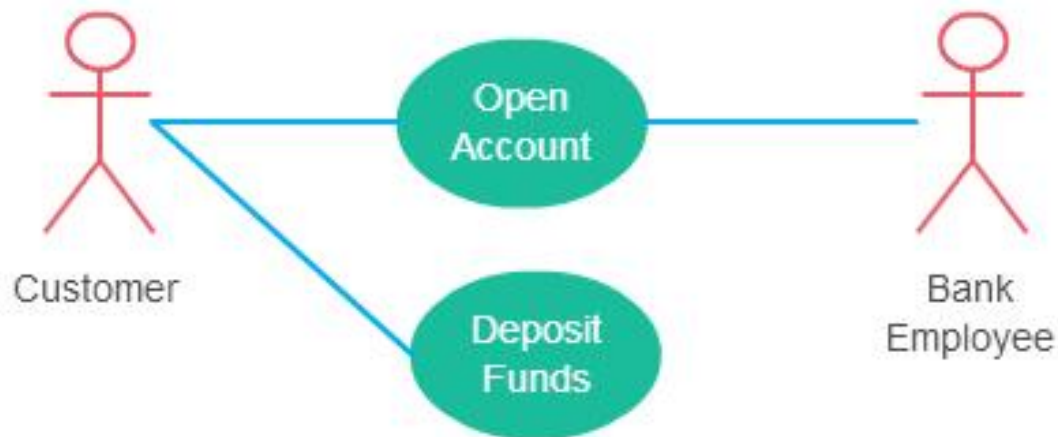
another optional element that is extremely useful in complex diagrams. Similar to class diagrams, packages are used to group together use cases.

USE CASE (CONT'D)

RELATIONSHIPS IN USE CASE DIAGRAMS

✓ There are five types of relationships in a use case diagram:

1. Association between an actor and a use case
2. Generalization of an actor
3. Extend relationship between two use cases
4. Include relationship between two use cases
5. Generalization of a use case



This one is straightforward and present in every use case diagram. Few things to note.

1. An actor **must be associated with at least one use case.**
2. An actor can be associated with multiple use cases.
3. Multiple actors can be associated with a single use case.

USE CASE (CONT'D)

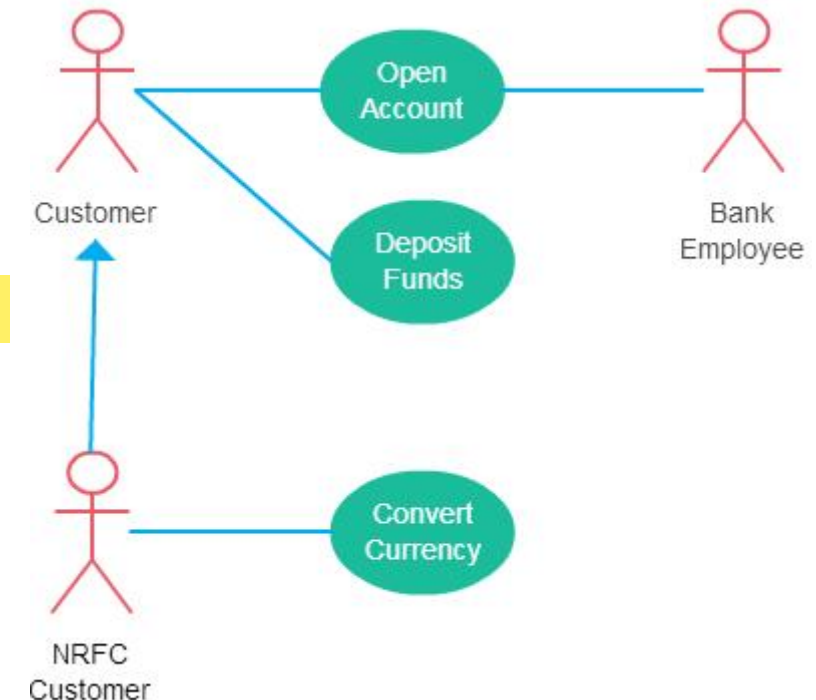


RELATIONSHIPS IN USE CASE DIAGRAMS

✓ There are five types of relationships in a use case diagram:

1. Association between an actor and a use case
2. **Generalization of an actor**
3. Extend relationship between two use cases
4. Include relationship between two use cases
5. Generalization of a use case

1. Generalization of an actor means that one actor can inherit the role of the other actor.
2. The descendant inherits all the use cases of the ancestor.
3. The descendant has one or more use cases that are specific to that role



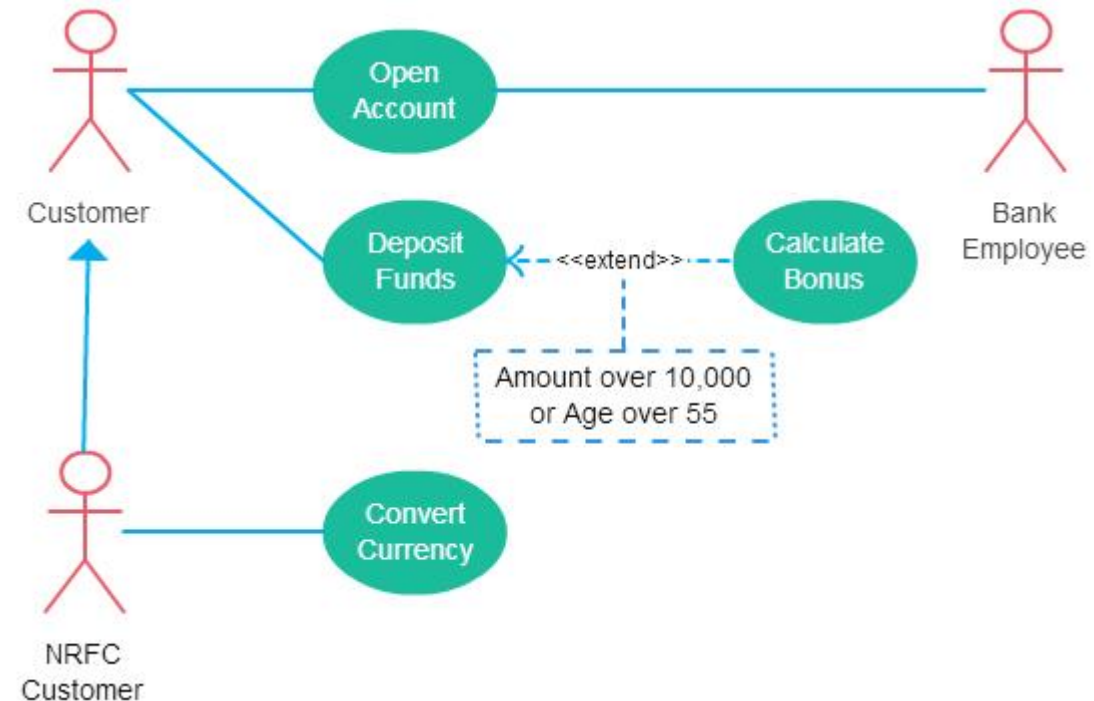
USE CASE (CONT'D)

RELATIONSHIPS IN USE CASE DIAGRAMS

✓ There are five types of relationships in a use case diagram:

1. Association between an actor and a use case
2. Generalization of an actor
3. **Extend relationship between two use cases**
4. Include relationship between two use cases
5. Generalization of a use case

1. it extends the base use case and adds more functionality to the system.
2. The extending use case is dependent on the extended (base) use case. .
3. The extending use case is usually optional
4. The extended (base) use case must be meaningful on its own



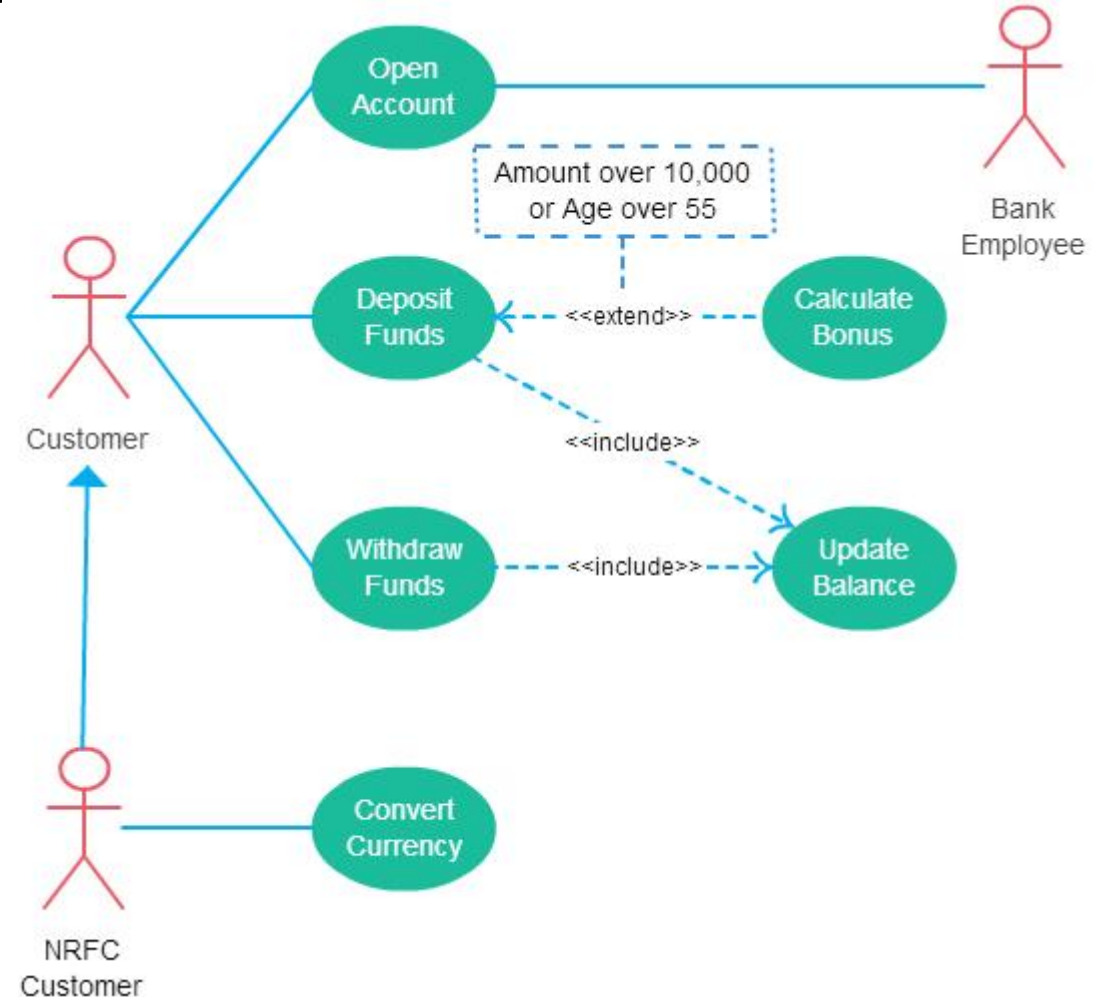
USE CASE (CONT'D)

RELATIONSHIPS IN USE CASE DIAGRAMS

✓ There are five types of relationships in a use case diagram:

1. Association between an actor and a use case
2. Generalization of an actor
3. Extend relationship between two use cases
4. **Include relationship between two use cases**
5. Generalization of a use case

1. show that the behavior of the included use case is part of the including (base) use case
2. The base use case is incomplete without the included use case.
3. The included use case is mandatory and not optional.

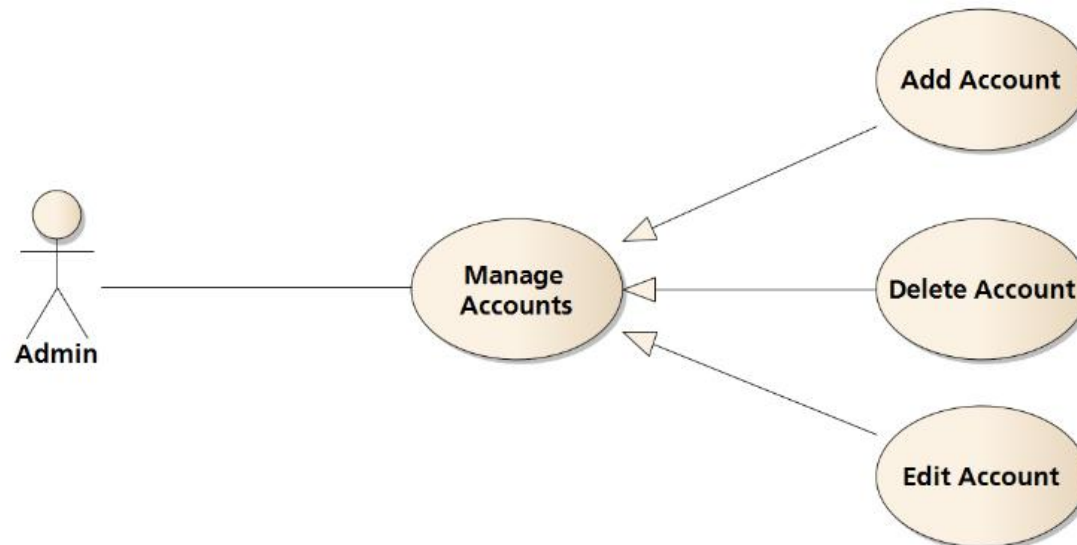


USE CASE (CONT'D)

RELATIONSHIPS IN USE CASE DIAGRAMS

✓ There are five types of relationships in a use case diagram:

1. Association between an actor and a use case
2. Generalization of an actor
3. Extend relationship between two use cases
4. Include relationship between two use cases
5. **Generalization of a use case**



1. This is similar to the generalization of an actor.
2. This is used when there is common behavior between two or more use cases and also specialized behavior specific to each use case.

USE CASE (CONT'D)



HOW TO CREATE USE CASE DIAGRAM

- Identifying Actors
- Identifying Use Cases
- Look for Common Functionality to use Include
- Is it Possible to Generalize Actors and Use Cases
- Optional Functions or Additional Functions

EXERCISE

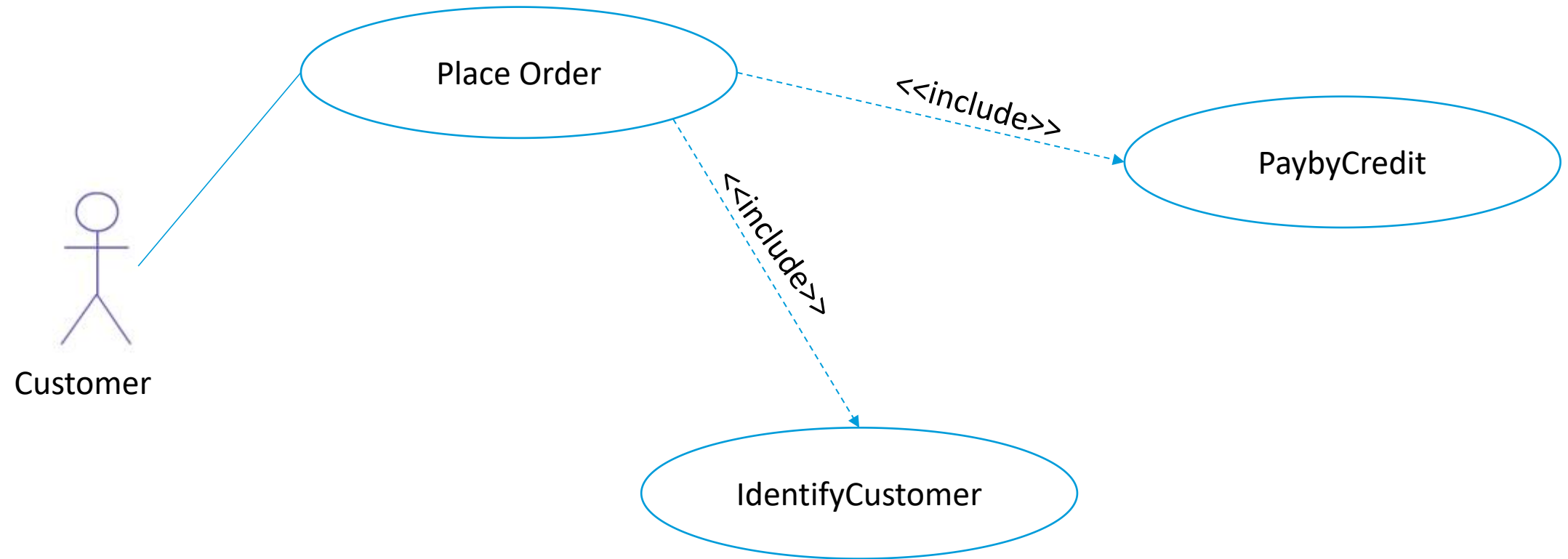


Pizza Hut Ordering System allows the customer to order pizza for home delivery through their website. To place an order, a customer searches to find items to purchase, adds items one at a time to a shopping cart, and possibly searches again for more items.

When all items have been chosen, the customer provides a delivery address. If not paying with cash, the customer also provides credit card information. The system has an option for customer to register with the Pizza Hut. They can then save their name and address so that they do not have to enter this every time they place an order.

Draw a use case diagram for a use case for placing an order, **PlaceOrder**. The use case should show a relationship to two previously defined use cases, **IdentifyCustomer** which allow a user to register and log in , and **PaybyCredit**, which models credit card payments.

SOLUTION



USE CASE (CONT'D)



USE CASE DESCRIPTION

Common fields present in a use case description:

- ✓ Use Case Name
- ✓ Use Case ID or Identifier
- ✓ Actors
- ✓ Preconditions
- ✓ Main Flow of Events
- ✓ Alternate Flows
- ✓ Exceptions
- ✓ Postconditions
- ✓ Business Rules
- ✓ Include Relationships (if applicable)
- ✓ Extend Relationships (if applicable)
- ✓ Frequency of Use
- ✓ Assumptions
- ✓ Dependencies
- ✓ Priority

USE CASE (CONT'D)



USE CASE DESCRIPTION : EXAMPLE

- ✓ **Use Case Name** : Purchase Product
- ✓ **Use Case ID or Identifier** : UC-01
- ✓ **Actors**
 - Customer (Primary Actor)
 - Online Shopping System (Secondary Actor)
- ✓ **Preconditions**
 - The customer is logged in to their account.
 - The customer has selected a product to purchase.
 - The product is available in the inventory.
- ✓ **Main Flow of Events**
 1. The customer selects a product to purchase.
 2. The system calculates the total price, including taxes and shipping costs.
 3. The customer chooses to proceed to checkout.
 4. The system prompts the customer for shipping and payment information.
 5. The customer provides shipping and payment details.
 6. The system verifies the payment information.
 7. If the payment is successful:
 - a) The system deducts the product's price from the customer's account.
 - b) The system updates the product's inventory.
 - c) The system generates an order confirmation.
 8. The system displays the order confirmation to the

✓ **Alternate Flows**

- ✓ Alternate Flow 1 – apply discount code:
 - In step 3 of the main flow, if the customer has a discount code:
 - The customer enters the discount code.
 - The system verifies the code's validity.
 - If the code is valid, the system applies the discount to the total price.
 - The process continues with step 4 of the main flow.

✓ **Exceptions**

- ✓ In step 2, if the selected product is out of stock:
 - The system displays an "Out of Stock" message.
 - The customer can choose a different product or cancel the purchase.
 - The use case continues from step 1.

✓ **Postconditions**

- The customer's order is recorded in the system's database.
- The product inventory is updated.
- The customer's account balance is adjusted.
- An order confirmation is sent to the customer's email.

✓ **Business Rules**

- Shipping costs are calculated based on the

USE CASE (CONT'D)

USE CASE DESCRIPTION : EXAMPLE

✓ Include Relationships (if applicable)

- includes the "View Product Details" use case (UC-02) to allow customers to view product details before purchasing.

✓ Extend Relationships (if applicable)

- can be extended by "Apply Discount Code" (UC-03) if the customer has a discount code.

✓ Frequency of Use

- Very high

✓ Assumptions

- Customers have valid login credentials.
- Payment processing services are available and reliable.
- Product details and prices are up-to-date.

~~✓~~ Dependencies

- This use case relies on the "View Product Catalog" feature to display available products.

✓ Priority

- High priority, as it represents a core functionality of the online shopping system

USE CASES ARE USEFUL TO...



- ✓ **Understanding User Needs:** emphasize the needs and goals of users, ensuring that the system is designed with the end user in mind.
- ✓ **Detailed Specifications:** provide a detailed description of functional requirements, which helps clarify what the system should do.
- ✓ **Visual Representation:** provide a visual representation of system functionality, making it easier for non-technical stakeholders to grasp the system's behavior.
- ✓ **Guidance for Development:** guide developers by providing clear instructions on what functionality needs to be implemented.
- ✓ **Test Case Development:** can be directly transformed into test cases, help to ensure that the developed system meets user requirements.
- ✓ **Handling Exceptions:** often include alternative flows and exception handling, ensuring that the system can gracefully handle unexpected situations.

CLASS DIAGRAMS



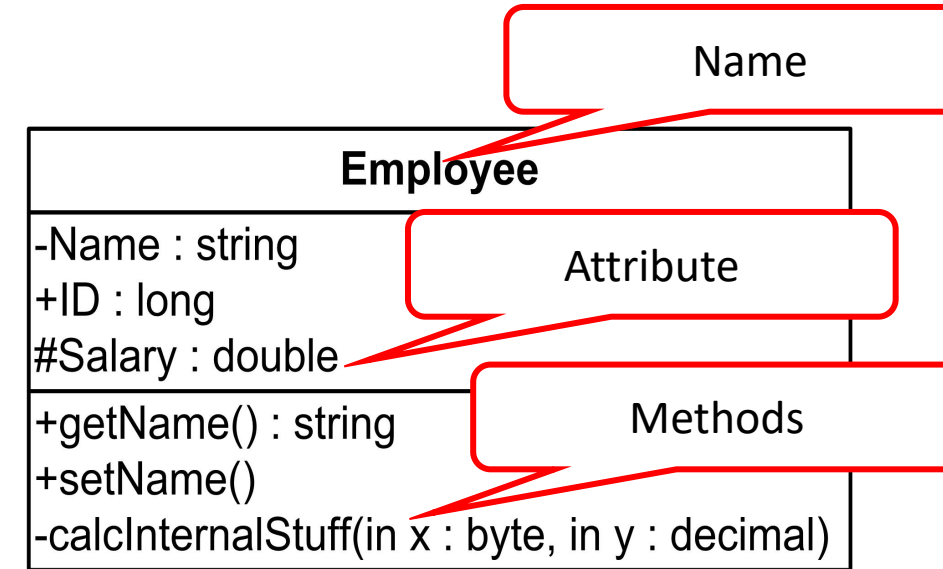
UML class diagram: a picture of

- the classes in an OO system
- their attributes and methods
- connections between the classes
 - that interact or inherit from each other
- Not represented in a UML class diagram:
 - details of how the classes interact with each other
 - algorithmic details; how a particular behavior is implemented

UML CLASS NOTATION



- A class is a rectangle divided into three parts
 - Class name
 - Class attributes (i.e. data members, variables)
 - Class operations (i.e. methods)
- Access Modifiers
 - Private: -
 - Public: +
 - Protected: #



attributes (fields, instance variables)

modifier name : type = default_value

attribute example:

#balance : double = 0.00

operations / methods

modifier name (parameters) : return_type

attribute example:

+ distance(p1: Point, p2: Point): double

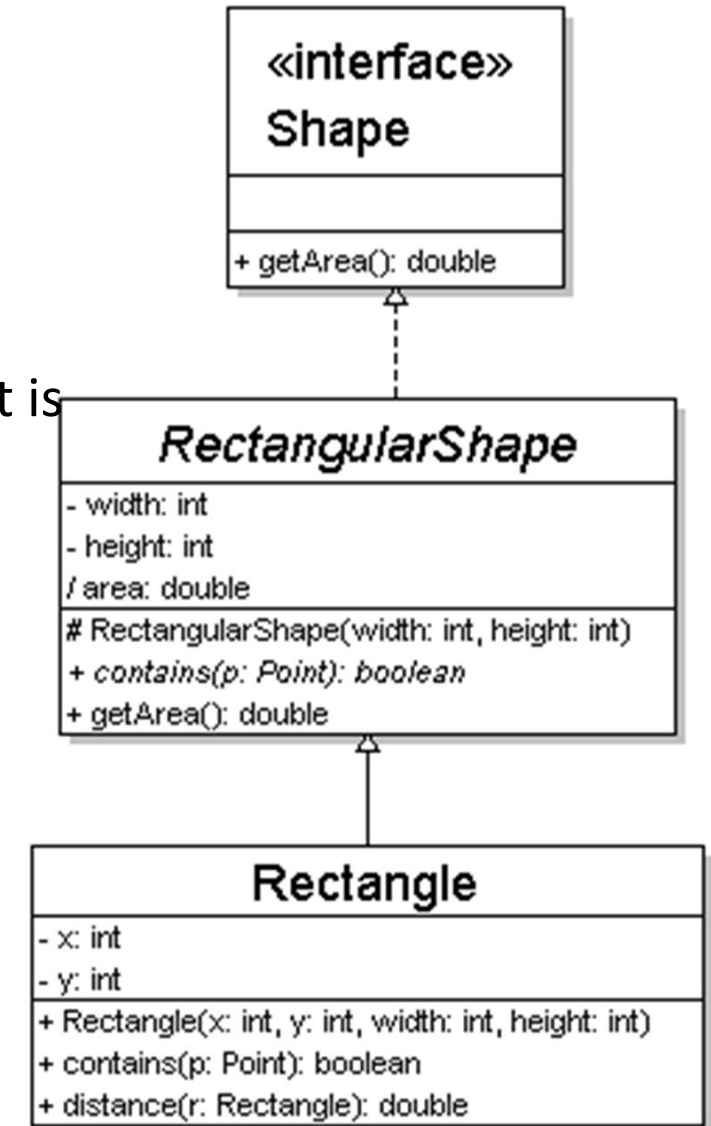
CLASS DIAGRAM RELATIONSHIPS



Generalization: **Inheritance** relationship

Association: **Usage** Relationship

- Dependencies
 - Aggregation
 - Composition
- hierarchies drawn top-down
- arrows point upward to parent
- line/arrow styles indicate whether parent is
- class:
solid line, black arrow
 - abstract class:
solid line, white arrow
 - interface:
dashed line, white arrow

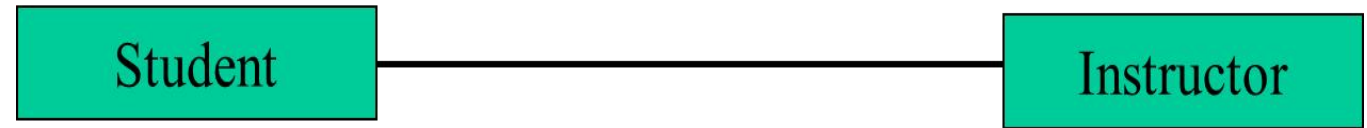


RELATIONSHIP

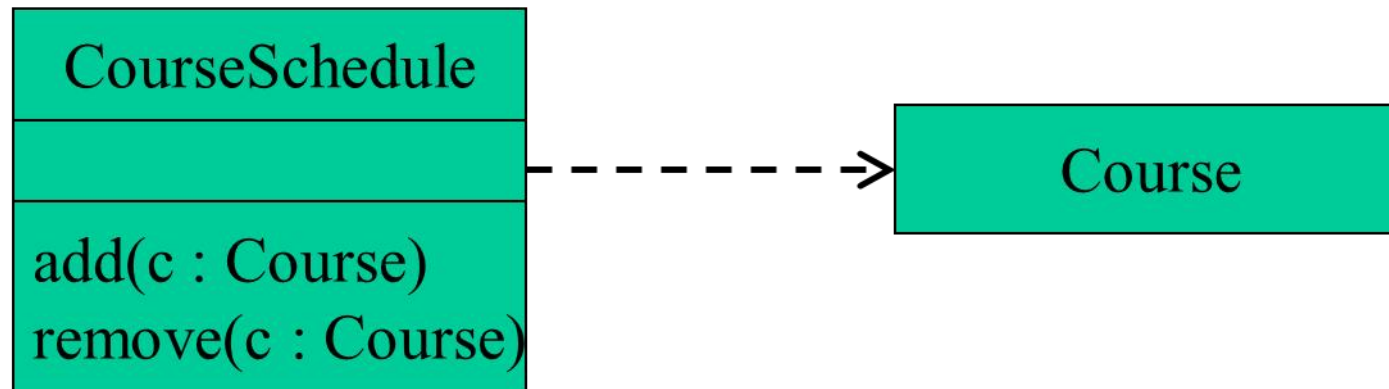
Association: Usage Relationship

- Dependencies
- Aggregation
- Composition

✓ If two classes in a model need to communicate with each other, there must be link between them. An association denotes that link.



A *dependency* indicates a relationship between two or more elements. The dependency from *CourseSchedule* to *Course* exists because *Course* is used in both the **add** and **remove** operations of *CourseSchedule*



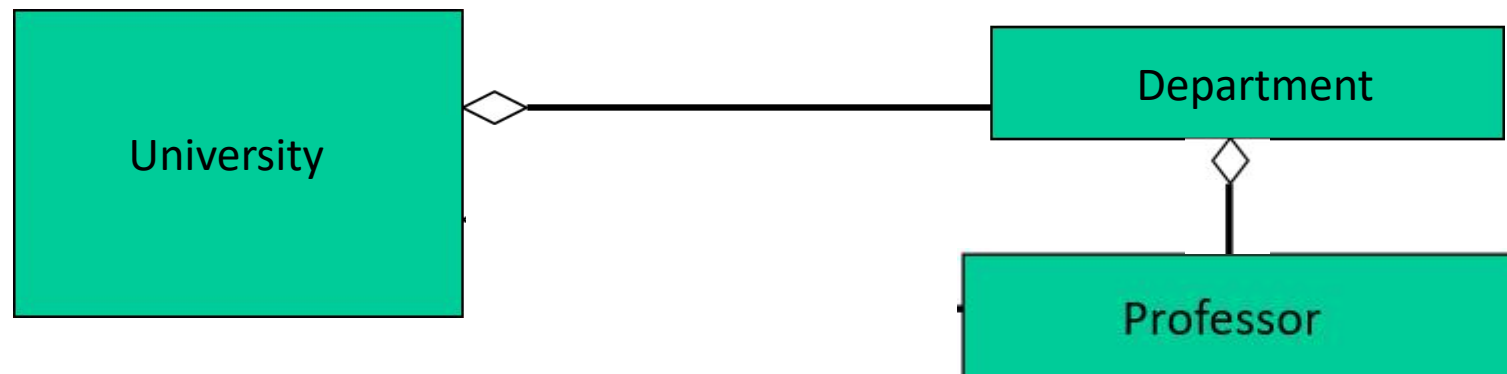
RELATIONSHIP



Association: Usage Relationship

- Dependencies
- **Aggregation**
- Composition

Aggregation represents a "*has-a*" relationship between objects where the **child object** can exist independently of the parent object. A hollow-diamond decoration on the association denotes aggregations.



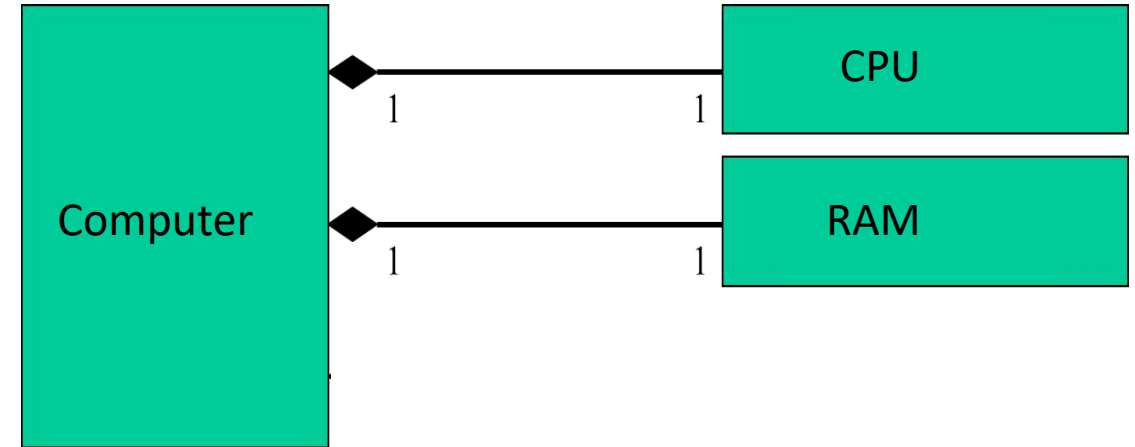
RELATIONSHIP

Association: Usage Relationship

- Dependencies
- Aggregation
- **Composition**

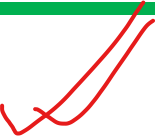
composition: represents a **stronger "has-a"** relationship where the **child object cannot exist independently of the parent object**. The child object's lifecycle is tied to that of the parent.

Consider a class House that contains Rooms. The rooms cannot exist without the house—if the house is destroyed, the rooms are destroyed too.



- Composition implies a strong ownership and lifecycle dependency.
- If the Computer object is no longer needed or is destroyed, its CPU and RAM components will also be disposed of.

COMPOSITION VS AGGREGATION



Aspect	Aggregation	Composition
Relationship Strength	Weak association ("has-a")	Strong association ("contains-a")
Object Independence	Child object <u>can exist independently</u>	Child object <u>cannot exist</u> without parent
Lifespan	Child's lifespan is independent of parent	<u>Child's lifespan depends on the parent</u>
Example	Library and Books	House and Rooms

Misidentifying Composition as Aggregation

Misidentifying Aggregation as Composition

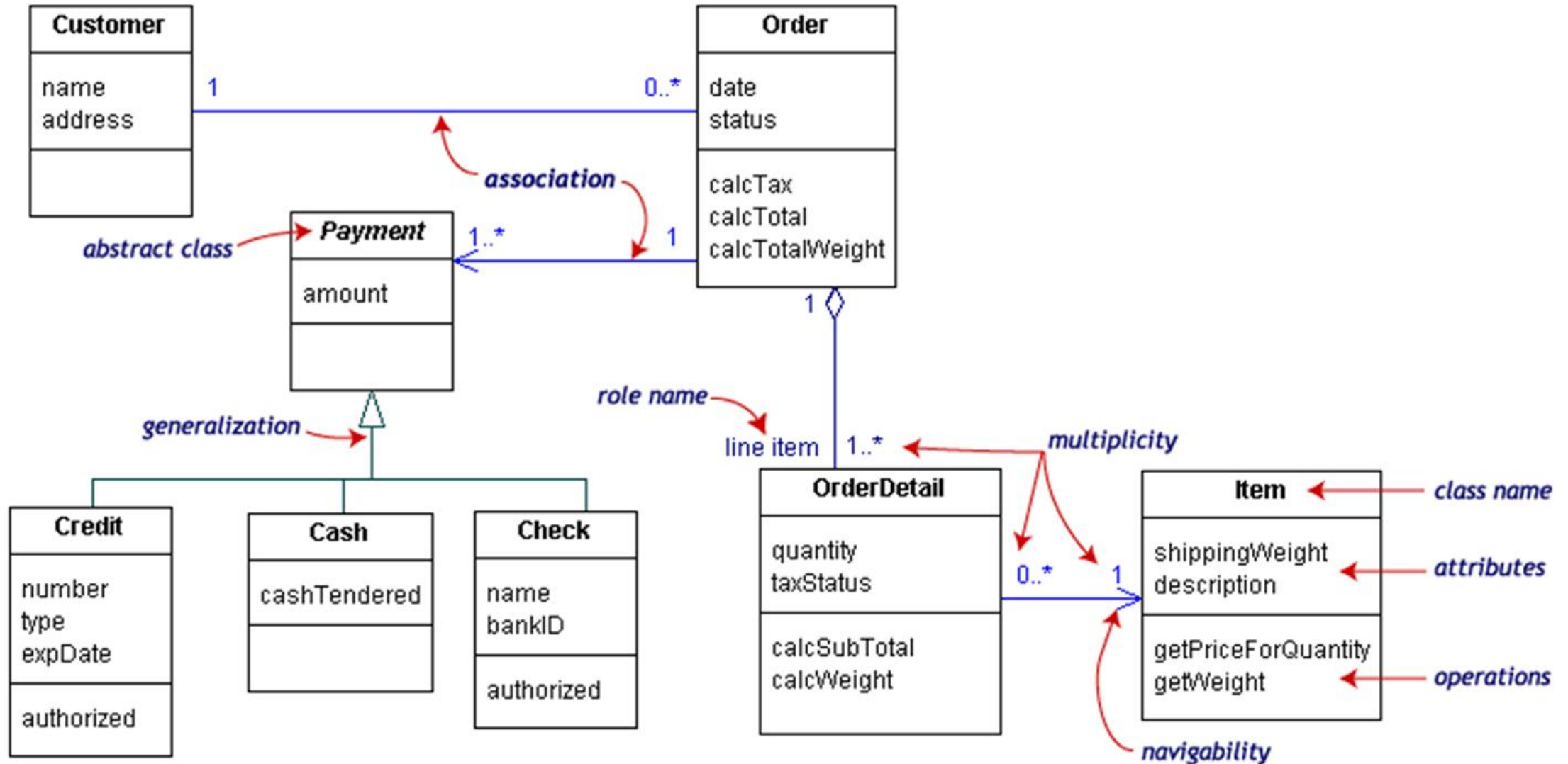
UML MULTIPLICITIES



Links on associations to specify more details about the relationship

Multiplicities	Meaning
0..1	zero or one instance. The notation <i>n</i> . . <i>m</i> indicates <i>n</i> to <i>m</i> instances.
0..* or *	no limit on the number of instances (including none).
1	exactly one instance
1..*	at least one instance

UML CLASS DIAGRAM EXAMPLE



EXERCISE



You are assigned to design a class diagram of a basketball tournament at your University.

- The tournament has 10 teams. Each team consist of 12 players.
- Each player has a specific height, speed, and accuracy.
- Players know which team they belong to.
- Some players are scholarship players. Scholarship players need to record their current grade-point average.
- Players may be transferred between teams.
- Teams play basketball games against other teams in the tournament.

SOLUTION



- The tournament has 10 teams. Each team consist of 12 players.
- Each player has a specific height, speed, and accuracy.
- Players know which team they belong to.
- Some players are scholarship players. Scholarship players need to record their current grade-point average.
- Players may be transferred between teams.
- Teams play basketball games against other teams in the tournament

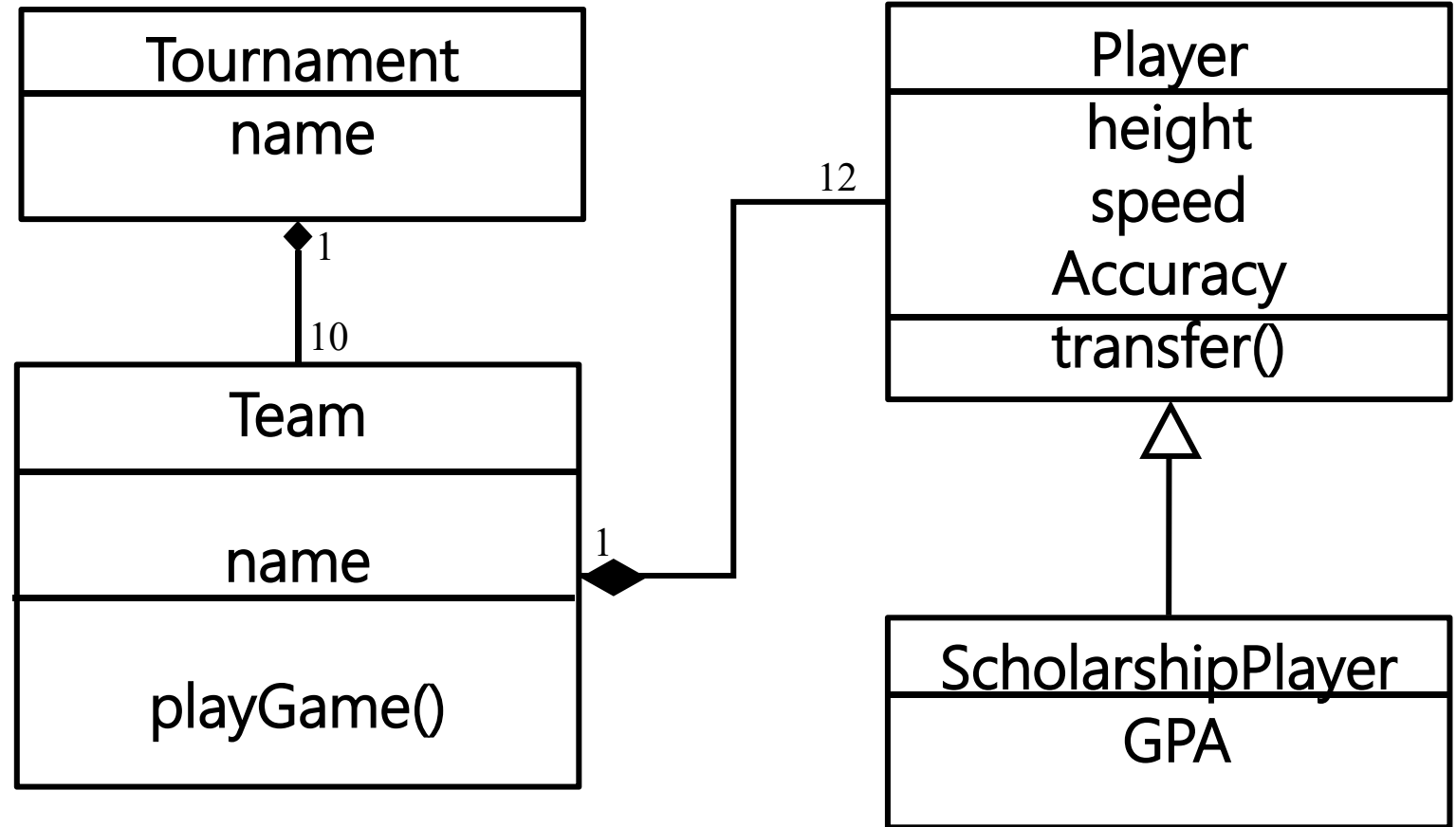
Class:

Tournament

Team

Player

ScholarshipPlayer



EXERCISE



Draw a UML class diagram to capture the following situation: “Every student is enrolled in a course. Each student may be enrolled in a set of units. Some units are core units for one or more courses and some units are elective units for one or more courses.”

Assumptions: a Course must have at least one Student enrolled in it and, as every student is enrolled in a Course, the relationship between Course and Student is 1-to-many. Assuming that elective units can be shared by different courses, between Course and ElectiveUnit is a many-to-many relationship (Similarly between Course and CoreUnit).

OBJECT DIAGRAM

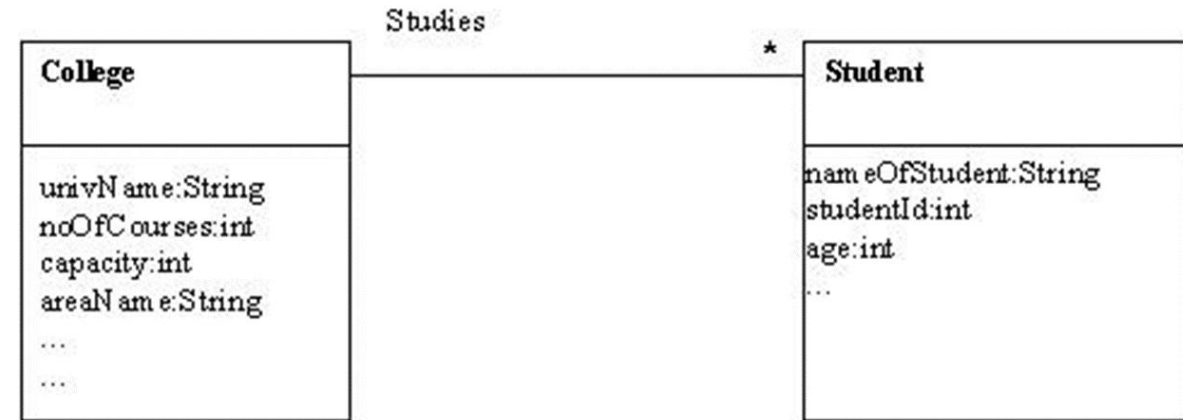


- In a live application classes are not directly used, but **instances or objects of these classes are used**.
- A pictorial representation of the relationships between these instantiated classes at any point of time (called objects) is called an "**Object diagram**."
- It looks very similar to a class diagram, and uses the similar notations to denote relationships.
- It reflects the picture of **how classes interact with each others at runtime** and in the actual system, how the **objects created at runtime are related to the classes**.
- shows this relation between the instantiated classes and the defined class, and the relation between these objects.

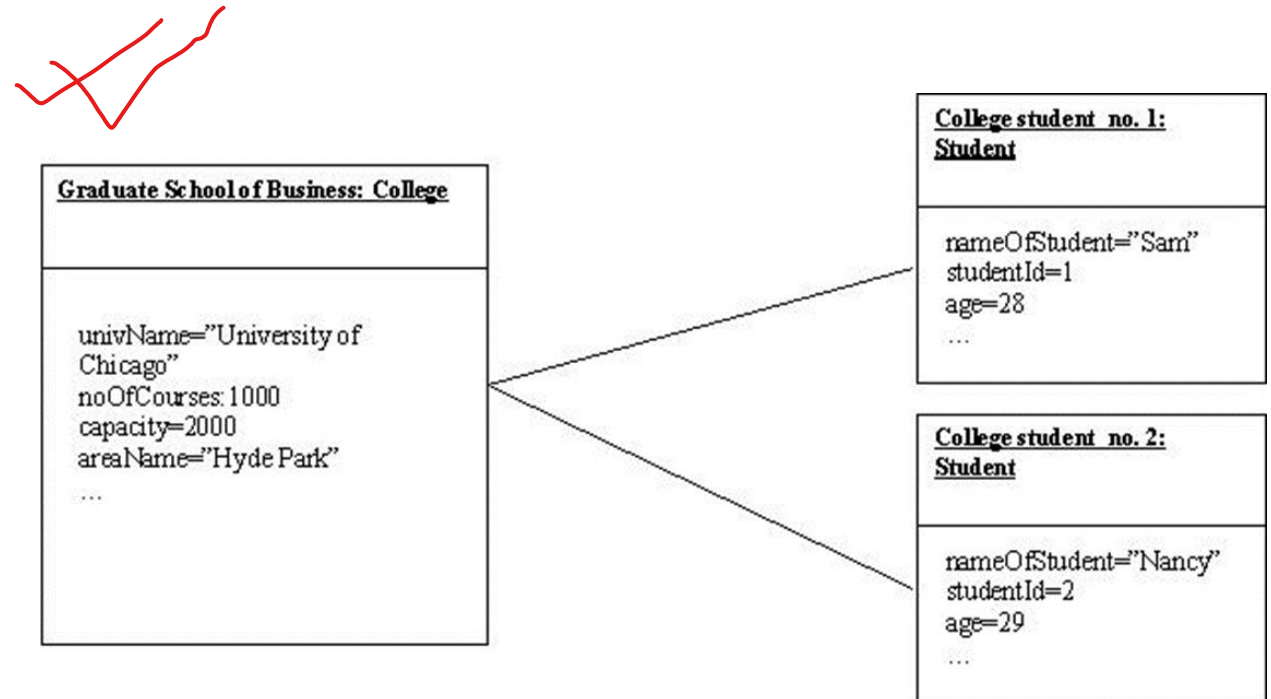
OBJECT DIAGRAM (CONT'D)



- ✓ the class diagram shows a class with attributes and methods declared.
- ✓ object diagram, these **attributes and method parameters are allocated values**.



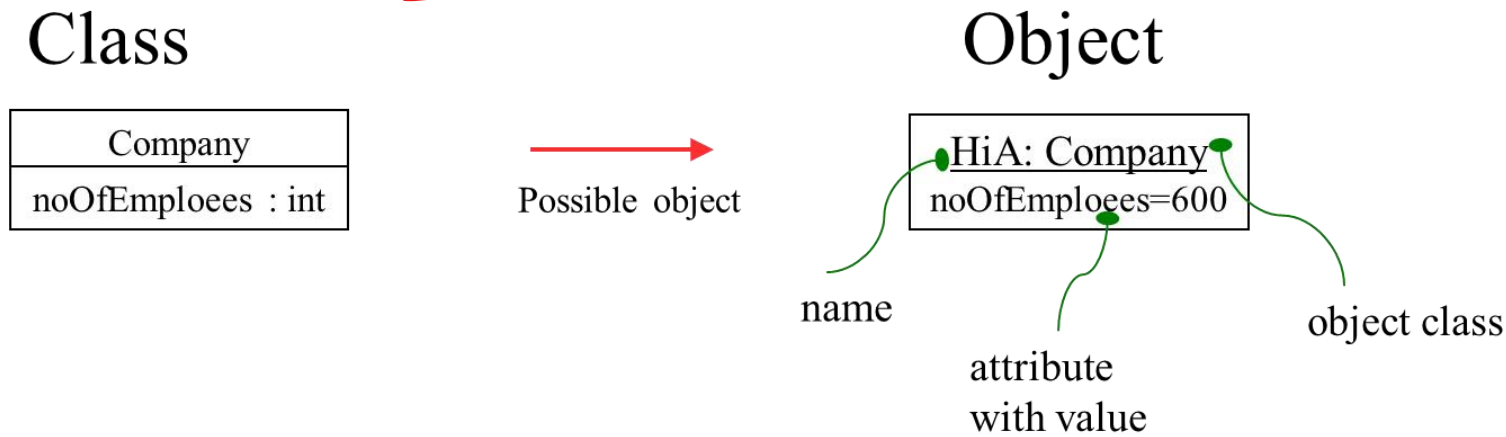
Now, when an application with the class diagram as shown above is run, instances of College and Student class will be created, with values of the attributes initialized. The object diagram for such a scenario will be represented as shown below:



OBJECT DIAGRAM (CONT'D)



The object diagram shows the **name of the instantiated object**, separated from the class name by a **":"**, and **underlined**, to show an instantiation.



When to use object diagrams?

- ✓ **Debug System Functionality:** They help visualize the relationships and attributes of objects in your system, making it easier to identify issues and debug functionality.
- ✓ **Verify Design Compliance:** Object diagrams allow you to check if the system has been designed according to requirements and if it behaves as expected in response to business needs.

Be careful !!

Avoid representing all the objects of your system in an object diagram → complex → unreadable.